

Real-Time Virtual Mentor for Introductory Programming Feedback

Alimzhan Koshikov

Master Degree Student, School of Software Engineering

Astana IT University (AITU)

Astana, Kazakhstan

225148@astanait.edu.kz

0009-0003-3234-6487

Tutkyshbayeva Shyryn

PhD, School of Software Engineering

Astana IT University (AITU)

Astana, Kazakhstan

sh.turkyshbayeva@astanait.edu.kz

0000-0002-1419-2571

Abstract—Introduction in programming is often challenging experience for CS students, especially for first-year students. Frequently introductory programming courses known as CS1 suffer from generic and delayed feedback and because of it students repeatedly face various errors (syntax, logic) without any guess about ways of resolving such errors. In order to solve this problem this paper proposes solution - virtual mentor for CS1 that provides real-time, tailored, personalized feedback which provides it during problem-solving session. The proposed virtual mentor provides real-time code analysis, lightweight student model and a pedagogical policy layer. Such combination can provide detection of common error patterns, track repeated mistakes and consider previous feedback from the history, and finally control when and how much guidance student should receive in order to improve their independence and critical thinking. Compared to other assistants with no constraints, the virtual mentor decides what type of feedback is suitable for particular student (for example, guiding questions, conceptual hints, code highlighting, or no feedback) and do it in minimal latency. In a pilot log-based evaluation, the system was keeping interactive responsiveness during the whole pilot session (mean end-to-end latency 741 ms, 809 ms at P95) and achieved an event-level success rate of 66.7% with a median time-to-fix of 2.45 s. These results show that real-time personalization with explicit pedagogical constraints can provide scalable support in CS1 settings.

Index Terms—Virtual mentor, CS1, introductory programming, programming education, real-time feedback, student modeling, pedagogical constraints

I. INTRODUCTION

Introductory programming (CS1) remains difficult for first-year students because they must simultaneously learn syntax, build semantic understanding, and develop problem decomposition skills. In large groups, beginners often cycle through compiler and logic errors via trial-and-error and receive limited individualized help, making timely formative feedback essential; yet providing it consistently at scale is constrained by instructor and TA capacity [1], [2].

Automated assessment improves scalability through unit tests, static checks, and compiler diagnostics, but the feedback is frequently binary or generic and does not address misconceptions or appropriate scaffolding. Computing education studies emphasize that feedback effectiveness depends on pedagogical method, timing, and whether guidance preserves student autonomy rather than simply enabling task completion

[1], [3]. Large-scale AI tutoring deployments confirm the feasibility of near real-time support: CS50 (introductory course on computer science taught at Harvard University) reported that by mid-November 2024 about 211,000 learners used their AI chatbot, generating roughly 10 million queries at an average cost of \$1.50 per student per year [4]. However, maintaining pedagogical alignment and reliability requires ongoing evaluation [4], and randomized CS1 studies show that higher perceived feedback quality does not consistently translate into improved performance, partly due to over-reliance and unverified inaccuracies [2]. Similarly, classroom evidence on LLM-driven assistants highlights immediate interactive help but also risks of erroneous or fabricated explanations and reduced conceptual engagement without safeguards [3], [5]. Prior work therefore argues that educational assistants should promote constructive help-seeking and avoid direct solution disclosure [1], [6], [7].

These results expose a gap: CS1 learners need real-time feedback that is context-aware, personalized to recurring error patterns, and pedagogically constrained to protect autonomy and reduce harmful reliance. Although guardrailed systems exist [6], [7], many LLM-based assistants lack explicit learner modeling and do not adapt feedback granularity to individual trajectories, while intelligent tutoring systems often require heavy domain engineering or do not meet real-time interaction constraints [8]. This paper proposes and evaluates a *virtual mentor* for CS1 that integrates continuous code analysis, a lightweight student model capturing recurring misconceptions and feedback history, and a pedagogical control layer that regulates timing, style, and disclosure. In contrast to unconstrained conversational assistants, the mentor targets bounded-latency intervention and adaptive scaffolding aligned with evidence-based characteristics of effective digital teaching assistants [1], [6].

The main contributions of this work are as follows:

- a modular virtual mentor architecture combining real-time code analysis, lightweight student modeling, and pedagogical control for CS1 support;
- an adaptive feedback mechanism that personalizes content and granularity based on recurring error patterns while avoiding direct solution disclosure;

- an experimental evaluation with first-year students demonstrating improved task completion performance and reduced recurring errors compared to non-personalized automated feedback.

II. RELATED WORK

AI support in programming education is commonly studied through three lines of work: (A) intelligent tutoring systems (ITS), (B) automated feedback via program analysis, and (C) LLM-based assistants. We summarize each and position the proposed virtual mentor.

A. Intelligent Tutoring Systems for Programming

ITS personalize instruction by modeling learner knowledge and adapting feedback. Modern systems increasingly focus on augmentation and adaptive sequencing rather than full automation. ProgTutor, for example, combines adaptive sequencing with automated evaluation while keeping instructors in the loop [8]. A recurring limitation is interaction granularity: many ITS intervene at discrete steps or after submission, which restricts timely support during active code construction.

Our virtual mentor targets the in-process setting with continuous monitoring and earlier pedagogical intervention.

B. Automated Feedback Generation Systems

Automated feedback is frequently used in CS1 courses for scalability and usually by the use of unit tests, static analysis and compiler diagnostics. Although such type of feedback is simple, fast and effective for quick checks, but it often superficial and binary, and it does not consider student's misconceptions. Kumar et al. said that majority of such tools checks only for syntax or correctness of output rather than providing conceptual and tailored guidance, while reproducibility and adaptation remain open issues [9].

These limitations help understand that solution is to combine code analysis with student modeling and adaptive pedagogical policies, as done in our architecture

C. LLM-Based Programming Assistants

Compared to rule-based feedback systems, LLMs allow natural-language explanations and hints, provide higher flexibility. [3]. Assistants with pedagogical constraints (pedagogical control layer) such as CodeAid and Iris confirm that LLM-based assistance can be effective when it designed to stimulate student's independence and critical thinking, rather than providing solutions directly. [6], [7]. However, statistic received from classroom also emphasize behavioral risks when students often abuse assistants for immediate help instead of conceptual understanding of particular topic [5], and eventually leads to a negative effect for learners in terms of understanding topic because of overdependence and potential misalignment with course goals. [2]. At MOOC scale, CS50 emphasizes the need for continuous human feedback and systematic evaluation to maintain pedagogical alignment [4].

D. Research Gap and Positioning

Most of the existing approaches hardly implements three main requirements in one system: (i) real-time support during code writing, (ii) an explicit lightweight student model that captures repeated mistakes, and (iii) a pedagogical control mechanism/policy that controls and regulates feedback timing and granularity. Majority of LLM assistants act as external help tools and often lack systematic learner modeling, while ITS solutions provide personalization but may not scale well or function under real-time constraints. Additionally, surveys report limited reproducibility due to private datasets and note that LLM-based studies often rely on proprietary models with restricted transparency and customization; they also introduce hallucination risk in educational feedback [9].

Our virtual mentor resolves this problem by integrating continuous code analysis, a lightweight student model, and a pedagogical policy layer that controls disclosure and timing to provide personalized guidance that helps preventing harmful reliance in CS1 courses, enhances critical thinking and independence which eventually lead to correct conceptual understanding of particular course/topic.

Table I summarizes representative automated feedback systems and positions the proposed virtual mentor relative to prior approaches.

III. VIRTUAL MENTOR SYSTEM ARCHITECTURE

The virtual mentor is a bounded real-time decision pipeline that maps a stream of student code states to pedagogically constrained feedback actions. The architecture separates code analysis, student modeling, policy decision, and feedback realization to support modularity, interpretability, and latency control.

Unlike standalone LLM assistants used as external help tools, the mentor is embedded into the programming workflow and incrementally observes code edits. If LLM-based text generation is used, it is only a realization component; all outputs are constrained by the pedagogical policy and student model signals. This design follows evidence that effective AI teaching assistants must balance immediacy, personalization, and learner autonomy while avoiding direct solution disclosure [1], [6], and that formative feedback is most effective when delivered during problem-solving rather than after submission [3].

A. Data Flow

As shown in Fig. 1, code edits are captured in real time by an analyzer that performs syntactic checks, lightweight semantic/structural analysis, and detection of common novice errors; incremental analysis reduces latency while preserving instructional relevance [8]. Extracted error types and code features are passed to a profile manager that maintains a lightweight student state (recurring mistakes, feedback history, approximate progression), which can improve feedback relevance over non-personalized baselines [7], [11]. A feedback engine then selects and realizes an adaptive response, while a pedagogical control layer regulates timing and granularity to reduce over-reliance

TABLE I
COMPARATIVE ANALYSIS OF AUTOMATED PROGRAMMING FEEDBACK SYSTEMS

System	Technology	Feedback Type	Effectiveness Metric
AIF 3.0 [10]	Hybrid (Data-driven + Expert-based)	Progress indicators (0–100%)	Subgoal detection accuracy: 88%
CodeAid [6]	GPT-3.5-Turbo	Pseudocode generation and explanations	Technical correctness: 79%
Verifix [9]	Program analysis (CFA)	Verified repair suggestions	Feedback quality score: 3.8/5.0
DeepFix [9]	Machine learning (neural networks)	Syntax error correction	Full error correction rate: 27%
Proposed Model	Rule-based baseline + ML-learned policy + Student Modeling	Real-time personalized feedback with pedagogical constraints	Pilot results reported in Sec. V (Table II)

reported in classroom deployments of LLM-based tools [5], [6]. Feedback is returned under bounded response time to preserve interactivity.

B. Formal System Model

Let c_t be the student code state after a modification at time t . The analyzer

$$A : c_t \rightarrow e_t$$

maps c_t to an error type $e_t \in \mathcal{E}$, a finite set of novice error categories.

The student profile is a lightweight state vector

$$s_t = (n_{e_1}, n_{e_2}, \dots, n_{e_k}),$$

where n_{e_i} is the cumulative count of error type e_i observed up to t .

Given (e_t, s_t) , the pedagogical policy selects an action

$$\pi : (e_t, s_t) \rightarrow a_t,$$

where $a_t \in \mathcal{A}$ and $\mathcal{A} = \{\text{GQ}, \text{CH}, \text{HL}, \text{NF}\}$ denote *Guiding Question*, *Conceptual Hint*, *Code Highlight*, and *No Feedback*.

In the baseline configuration, π is rule-based. In the adaptive configuration, π is learned from interaction logs via supervised multi-class classification:

$$\hat{a} = \arg \max_{a \in \mathcal{A}} P(a | x),$$

where x includes the current error type, error recurrence counts, and interaction timing features.

The system does not train or fine-tune LLMs. Machine learning is applied only to policy learning; any language generation operates solely as constrained feedback realization.

C. Core Components

Analyzer: Continuously evaluates code via syntactic validation, structural checks, and novice error detection, consistent with evidence that automated debugging/error classification supports CS1 when pedagogically constrained [3].

Profile Manager: Maintains an interpretable, low-overhead student model (recurring errors, prior feedback exposure, approximate skill), prioritizing scalability over heavy knowledge tracing [8].

Feedback Engine: Combines analyzer outputs and student state to realize feedback (guiding questions, conceptual hints, code region highlighting) while restricting direct solution disclosure [1], [6].

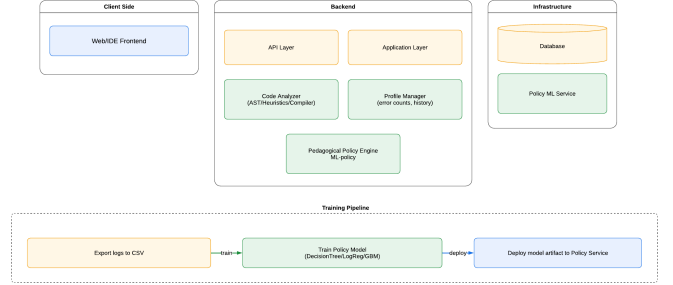


Fig. 1. High-level architecture of the virtual mentor system.

IV. REAL-TIME FEEDBACK MECHANISM

The real-time feedback mechanism is designed to support beginner learners during active problem-solving session rather than after task completion. Research on programming assistants which based on LLMs shows that delayed or premature and too detailed feedback can stimulate trial-and-error behavior and reduce cognitive engagement and can lead to superficial understanding because of over-reliance [5].

To solve this problem, feedback in the proposed system is generated gradually as students modify their code, allowing timely intervention when misconceptions or repeated errors are detected.

A. Feedback Generation

The generation of feedback in the proposed system passes through many stages. First, while a learner is writing code, the analyzer is searching and analyzing code for syntax and logic errors, and also identifying popular error patterns of beginners. Following this process, after the analyzer collected these results, the results are saved in the student profile, and the system defines whether it is a repeated error or a new problem.

Next, the feedback engine chooses an appropriate strategy, for example, providing a conceptual hint or explanation, asking a guiding question, or highlighting relevant parts of code. This approach implements ideas from previous works and it demonstrates that hints are more effective than direct solutions and such approach helps students improve their understanding and align course goals with adjusts student's learning trajectory with course objectives [1], [7].

Revealing solution directly is intentionally avoided to promote critical and independent thinking, which corresponds with statistic received from large-scale classroom integrations of LLM-based assistants [6].

B. Timing Constraints

In order to implement real-time interaction, feedback generation is bounded by a limited response time by using lightweight analysis techniques, incremental updates of the student model and it eventually ensures that proposed system delivers feedback immediately after code modification.

Such approach shows that low-latency feedback enhances student engagement, and meanwhile minimizing errors in programming work session [3], [8]. By avoiding heavy model inference during critical interaction moments, the system remains responsive and scalable in large courses.

To ensure real-time interaction, the total system response time is constrained by

$$T_{total} = T_{analysis} + T_{policy} + T_{feedback},$$

where $T_{analysis}$ denotes code analysis time, T_{policy} represents pedagogical decision time, and $T_{feedback}$ corresponds to feedback generation.

In the prototype of virtual mentor, all components works under limited latency and it done for providing feedback rapidly during a code writing process within suitable interactive time constraints for programming.

In pilot version, interaction logs showed the end-to-end latency remained within interactive time bounds, with mean $T_{total} = 741$ ms and P95 809 ms. Averages withing components were 618 ms for analysis, 26 ms for policy selection, and 12 ms for feedback realization.

V. EXPERIMENTAL EVALUATION

A. Dataset Description

Dataset was consisted of first-year undergraduate students enrollend introductory programming courses (CS1). Participants were randomly assigned to either a control group receiving non-personalized automated feedback or an experimental group using the virtual mentor system.

Both groups completed identical programming tasks under comparable instructional conditions. This experimental setup follows established evaluation practices in prior studies on AI-supported programming education [2], [7].

All student–system interactions were logged at the event level, including detected error types, selected feedback actions, and timing measurements. These interaction logs form a structured dataset used for training and evaluating the adaptive pedagogical policy.

Each interaction record is represented as

$$x_i = (e_i, c_i, t_i^{analysis}, t_i^{feedback}, s_i),$$

where e_i denotes the detected error type, c_i the recurrence count, $t_i^{analysis}$ and $t_i^{feedback}$ the measured latency components, and s_i the task outcome indicator.

TABLE II
PILOT RESULTS FROM INTERACTION LOGS (EVENT-LEVEL)

Metric	Value
Events (K)	60
Mean latency $\bar{T}$ (ms)	741
P95 latency (ms)	809
Mean analysis/policy/feedback (ms)	618 / 26 / 12
Platform overhead (ms)	85
Event success rate (%)	66.7
Median time-to-fix (ms)	2450
P95 time-to-fix (ms)	3595

TABLE III
SUCCESS RATE BY FEEDBACK ACTION (PILOT)

Action	N	Success rate
CODE_HIGHLIGHT	23	1.00
CONCEPTUAL_HINT	15	1.00
GUIDING_QUESTION	19	0.25
NO_FEEDBACK	3	0.00

B. Metrics

System effectiveness was evaluated using both performance and behavioral metrics. Error recurrence reduction was computed as

$$\Delta E = \frac{E_{baseline} - E_{mentor}}{E_{baseline}},$$

where E denotes the average number of repeated errors per task.

Additionally, average system response latency was measured as

$$\bar{T} = \frac{1}{N} \sum_{i=1}^N T_{total}^{(i)},$$

capturing the responsiveness of the virtual mentor during live interaction.

C. Results

Table II summarizes pilot outcomes computed from event-level interaction logs. We report (i) responsiveness of the real-time pipeline via latency statistics and (ii) feedback effectiveness proxies via event-level success and time-to-fix, where *success* denotes that the detected issue was resolved after the issued feedback action.

VI. DISCUSSION

A. Interpretation of Results

The pilot results (Table II) indicate that the virtual mentor can operate under interactive time constraints while providing effective event-level support. The end-to-end latency remained low (mean 741 ms; P95 809 ms), suggesting that the analysis–policy–feedback pipeline is suitable for real-time use during code construction. In addition, the system achieved an event-level success rate of 0.667 with a median time-to-fix of 2.45 s, indicating that the issued feedback actions frequently led to resolved issues.

Action-level breakdown (Table III) shows that conceptual hints and code highlighting were associated with higher immediate resolution rates than guiding questions in the pilot trace, which motivates further policy tuning and larger-scale evaluation.

These results confirm previous research statistic, and show that feedback efficiency depends not only on urgency but also on alignment with pedagogical methodologies and personalization [1], [3]. And by regulating feedback detailing and content, the virtual mentor can earlier detect errors and misconceptions and accurately interfere when it necessary, when student gets stuck while solving a problem.

The observed reduction in repeated errors may be indicative of the system promoting conceptual understanding rather than promoting trial-and-error behavior. This results align with findings from classroom studies of LLM-based assistants, which emphasize that hints and guiding questions are more effective than direct solution for supporting stable and effective learning track [6], [7]. Furthermore, regulating feedback to avoid revealing complete solutions apparently reduce superficial help-seeking patterns previously reported in studies of unlimited LLM-based programming assistants [5].

B. Limitations

This study has several limitations. First, the pilot covers a limited set of CS1 tasks and may not represent open-ended or design-oriented problems where feedback needs differ [8]. Second, outcomes are short-term: we did not measure long-term retention or transfer, which prior deployments highlight as essential for evaluating tutoring impact [2]. Third, despite pedagogical controls, some over-reliance may still occur, especially under time pressure, as reported in LLM-based classroom studies [1], [5]. Finally, the current student model is intentionally lightweight; richer models could capture more learning signals but introduce complexity, data demands, and privacy trade-offs [11].

C. Practical Implications

The findings indicate that real-time personalized feedback can be delivered without replacing instructors or TAs. With pedagogical control and lightweight student modeling, the mentor complements automated assessment and can reduce repetitive support load while maintaining instructional intent. This matches recommendations that AI tutors work best as supportive assistants rather than authoritative solvers [4], [6]. In practice, such a mentor can act as a middle layer between static automated checks and human tutoring, but it should be integrated into course design and monitored to ensure interaction patterns remain aligned with learning objectives [2].

D. Ethical Considerations

The system is designed to preserve student autonomy and responsible AI use by avoiding direct solution disclosure and prioritizing formative guidance [1]. Interaction logs are used only for educational analysis and are anonymized. Pedagogical

controls are included to reduce over-reliance, a recurring concern in studies of LLM-based educational tools [5], [6].

VII. FUTURE WORK

Future work focuses on four directions. First, we will run longitudinal studies to test whether the observed short-term gains translate into durable learning, including retention and transfer across topics, as prior work shows these outcomes may diverge in CS1 [2], [9].

Second, we will extend the student model beyond simple error counts to capture temporal learning dynamics (e.g., recurrence over time) and limited forms of knowledge tracing, while keeping the approach scalable, interpretable, and privacy-aware [11].

Third, we will add human-in-the-loop controls so instructors can inspect, calibrate, and refine feedback policies, which improves alignment with course-specific pedagogy in large deployments [4], [6].

Finally, we will evaluate generalizability across languages and paradigms (e.g., object-oriented and functional programming) to validate robustness beyond introductory procedural tasks.

VIII. CONCLUSION

A virtual mentor model for introductory programming was developed to deliver real-time personalized feedback through continuous code analysis, lightweight student modeling, and pedagogical control.

A pilot log-based evaluation demonstrated interactive responsiveness (mean end-to-end latency **741 ms**; P95 **809 ms**) and achieved an event-level success rate of **66.7%** with a median time-to-fix of **2.45 s**. The results indicate that bounded-latency, pedagogically constrained feedback pipelines are feasible as scalable support tools for CS1.

Future work will validate the approach on larger cohorts and under comparative baselines (e.g., non-personalized automated feedback) to quantify learning-related outcomes, including reductions in error recurrence and improvements in task-level success.

ACKNOWLEDGMENT

The authors thank the students and instructors who participated in the experimental evaluation. This research was conducted as part of the master's degree program at Astana IT University. The authors also acknowledge Astana IT University for providing the academic and computing infrastructure that supported this study.

REFERENCES

- [1] P. Denny, S. MacNeil, J. Savelka, L. Porter, and A. Luxton-Reilly, "Desirable characteristics for ai teaching assistants in programming education," p. 408–414, 2024. [Online]. Available: <https://doi.org/10.1145/3649217.3653574>
- [2] U. Z. Ahmed, S. Sahai, B. Leong, and A. Karkare, "Feasibility study of augmenting teaching assistants with ai for cs1 programming feedback," pp. 11–17, 2025. [Online]. Available: <https://doi.org/10.1145/3641554.3701972>

- [3] A. F. Pereira and R. Ferreira Mello, "A systematic literature review on large language models applications in computer programming teaching evaluation process," *IEEE Access*, vol. 13, pp. 113 449–113 460, 2025. [Online]. Available: <https://doi.org/10.1109/ACCESS.2025.3584060>
- [4] R. Liu, J. Zhao, B. Xu, C. Perez, Y. Zhukovets, and D. J. Malan, "Improving ai in cs50: Leveraging human feedback for better learning," p. 715–721, 2025. [Online]. Available: <https://doi.org/10.1145/3641554.3701945>
- [5] B. Sheese, M. Liffiton, J. Savelka, and P. Denny, "Patterns of student help-seeking when using a large language model-powered programming assistant," p. 49–57, 2024. [Online]. Available: <https://doi.org/10.1145/3636243.3636249>
- [6] M. Kazemitabaar, R. Ye, X. Wang, A. Z. Henley, P. Denny, M. Craig, and T. Grossman, "Codeaid: Evaluating a classroom deployment of an llm-based programming assistant that balances student and educator needs," 2024. [Online]. Available: <https://doi.org/10.1145/3613904.3642773>
- [7] P. Bassner, E. Frankford, and S. Krusche, "Iris: An ai-driven virtual tutor for computer science education," p. 394–400, 2024. [Online]. Available: <https://doi.org/10.1145/3649217.3653543>
- [8] J. Ortega-Morla, A. Leis, A. Mallo, L. Morán-Fernández, S. Guerreiro, A. Paz-López, B. Pérez-Sánchez, N. Sánchez-Marño, A. Rodríguez-Arias, O. Fontenla-Romero, and F. Bellas, "Progtutor: A robotic-based framework to support teaching and learning of programming fundamentals," *IEEE Transactions on Learning Technologies*, vol. 18, pp. 783–797, 2025. [Online]. Available: <https://doi.org/10.1109/TLT.2025.3598041>
- [9] S. S. Kumar, M. A. Lones, M. Maarek, and H. Zantout, "Navigating the landscape of automated feedback generation techniques for programming exercises," *ACM Trans. Comput. Educ.*, vol. 25, no. 4, Oct. 2025. [Online]. Available: <https://doi.org/10.1145/3764593>
- [10] S. Marwan, B. Akram, T. Barnes, and T. W. Price, "Adaptive immediate feedback for block-based programming: Design and evaluation," *IEEE Transactions on Learning Technologies*, vol. 15, no. 3, pp. 406–420, 2022. [Online]. Available: <https://doi.org/10.1109/TLT.2022.3180984>
- [11] R. Sajja, Y. Sermet, M. Cikmaz, D. Cwierny, and I. Demir, "Artificial intelligence-enabled intelligent assistant for personalized and adaptive learning in higher education," *Information*, vol. 15, no. 10, 2024. [Online]. Available: <https://www.mdpi.com/2078-2489/15/10/596>
- [12] R. Francisco and F. Silva, "Intelligent tutoring system for computer science education and the use of artificial intelligence: A literature review," p. 338–345, 2022. [Online]. Available: <http://dx.doi.org/10.5220/0011084400003182>
- [13] K. Chrysafiadi, M. Virvou, G. A. Tsihrintzis, and I. Hatzilygeroudis, "Evaluating the user's experience, adaptivity and learning outcomes of a fuzzy-based intelligent tutoring system for computer programming for academic students in greece," *Education and Information Technologies*, vol. 28, no. 6, p. 6453–6483, Nov. 2022. [Online]. Available: <http://dx.doi.org/10.1007/s10639-022-11444-3>
- [14] M. Pankiewicz and R. S. Baker, "Large language models (gpt) for automating feedback on programming assignments," 2023. [Online]. Available: <https://arxiv.org/abs/2307.00150>
- [15] S. Nayak, R. Agarwal, and S. K. Khatri, "Automated assessment tools for grading of programming assignments: A review," pp. 1–4, 2022. [Online]. Available: <https://doi.org/10.1109/ICCCI54379.2022.9740769>
- [16] J.-Y. Kuo, H.-C. Lin, P.-F. Wang, and Z.-G. Nie, "A feedback system supporting students approaching a high-level programming course," *Applied Sciences*, vol. 12, no. 14, 2022. [Online]. Available: <https://www.mdpi.com/2076-3417/12/14/7064>
- [17] I. Estévez-Ayres, P. Callejo, M. Hombrados-Herrera, C. Alario-Hoyos, and C. Delgado Kloos, "Evaluation of llm tools for feedback generation in a course on concurrent programming," p. 774–790, 2024. [Online]. Available: <https://doi.org/10.1007/s40593-024-00406-0>
- [18] T. Phung, V.-A. Pădurean, A. Singh, C. Brooks, J. Cambronero, S. Gulwani, A. Singla, and G. Soares, "Automating human tutor-style programming feedback: Leveraging gpt-4 tutor model for hint generation and gpt-3.5 student model for hint validation," p. 12–23, 2024. [Online]. Available: <https://doi.org/10.1145/3636555.3636846>
- [19] E. Frankford, C. Sauerwein, P. Bassner, S. Krusche, and R. Breu, "Ai-tutoring in software engineering education," p. 309–319, 2024. [Online]. Available: <https://doi.org/10.1145/3639474.3640061>
- [20] M. Messer, N. C. C. Brown, M. Kölling, and M. Shi, "Automated grading and feedback tools for programming education: A systematic review," *ACM Trans. Comput. Educ.*, vol. 24, no. 1, Feb. 2024. [Online]. Available: <https://doi.org/10.1145/3636515>